The Last Clue

Checkpoint 3: OpenGL Integration + Playable Prototype

Our algorithms draw every frame; OpenGL displays it and draws overlays; GLFW gives the window and input.

Group members - CP3 work

- Caleb Kipchirchir - GLFW window + loop, Canvas->OpenGL bridge, ClueScript interpreter, prototype integration
- Sudheysi Ali - game state machine, picking/dragging, interactive letter + safe puzzles, code-puzzle checking
- Latiffa - text layout + buttons, in-game code editor and laptop, live Case File, interactive evidence board
- Alvin Kimani - playable crime scene: hotspots, walking detective, laptop, painting close-up

Play it

https://github.com/C-Sawe/the-last-clue - cmake -B build -S . && cmake --build build && ./build/the_last_clue

1. The game in its OpenGL window (real screenshot)

Caleb (window, bridge) + Alvin (scene)



Read back from OpenGL with glReadPixels after real mouse input: the title was clicked away and the mouse hovers the safe. Our algorithms drew the frame into a Canvas; OpenGL shows it as a texture and draws the hover outline with GL_LINE_LOOP. GLFW provides the window and input.

2. First OpenGL program: scene viewer

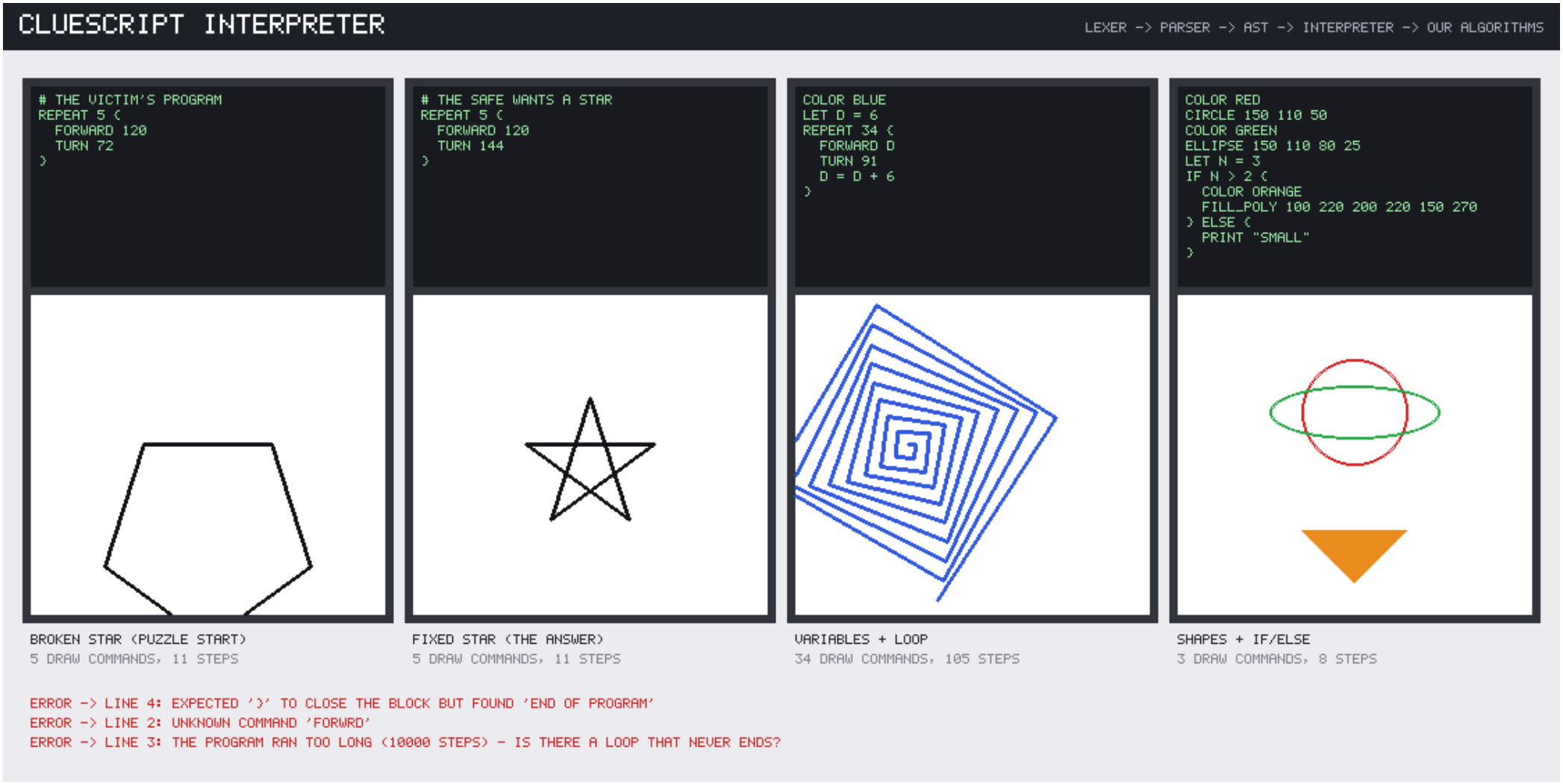
Caleb



Canvas -> texture upload each frame (glTexSubImage2D), letterboxed textured quad with an orthographic projection in canvas coordinates; the yellow cursor box is an OpenGL line primitive.

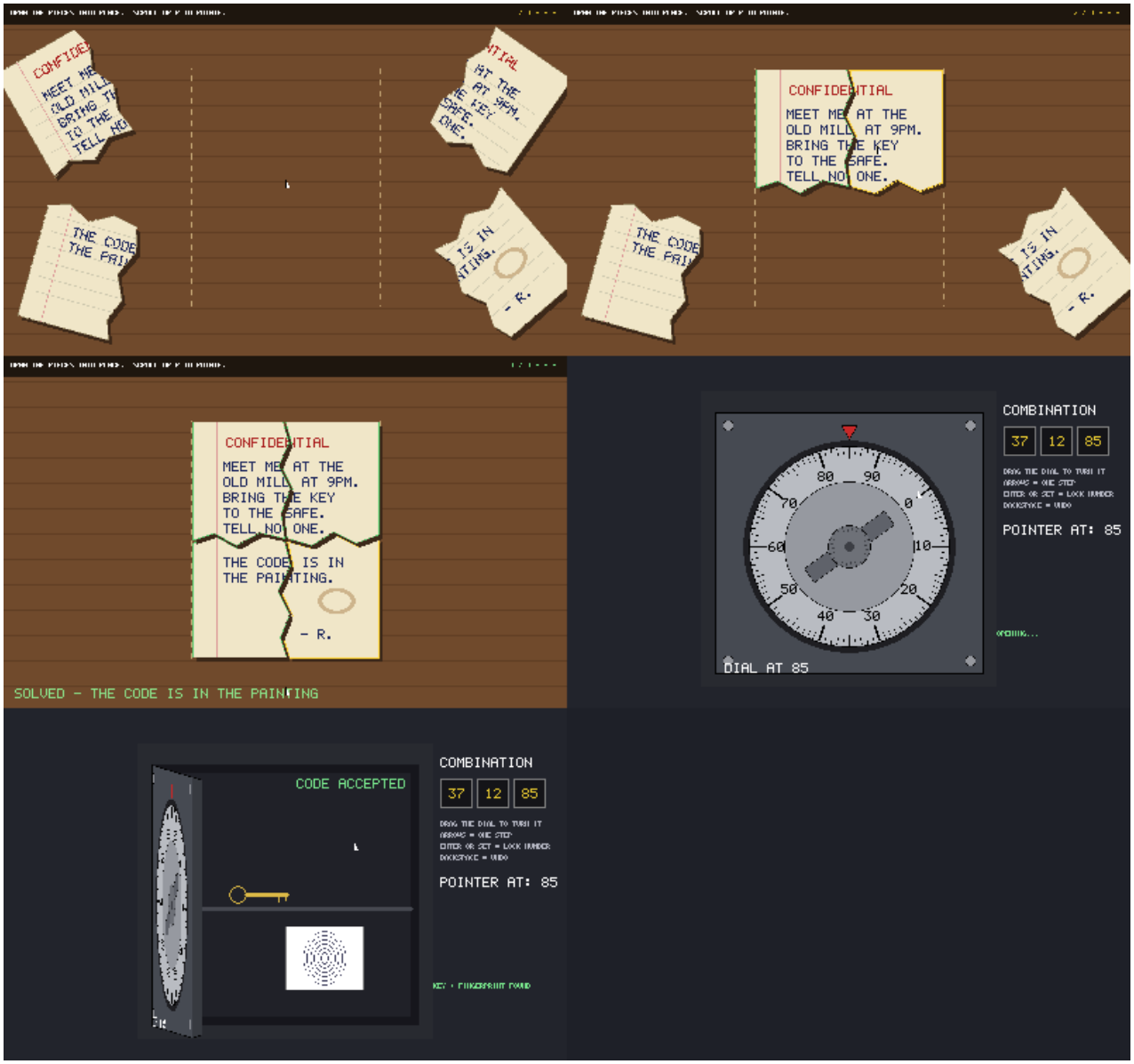
3. ClueScript interpreter

Caleb



Lexer -> recursive-descent parser -> AST -> tree-walking interpreter with a step limit. The turtle's 'forward' rotates the step vector by the heading (CP1 rotation matrix). Output drawn with Bresenham / midpoint / scanline and clipped to each monitor. Friendly errors with line numbers.

4. Interactive puzzles (scripted mouse input)



Letter: pieces picked with point-in-polygon on their transformed shapes, dragged with a grab offset and clamped to the desk, rotated 15 degrees per wheel notch, snapped when within 16 px and ~11 degrees. Safe: the dial follows the change in the mouse's atan2 angle; Enter locks numbers; 37-12-85 opens the door.

5. Checking a code-puzzle answer

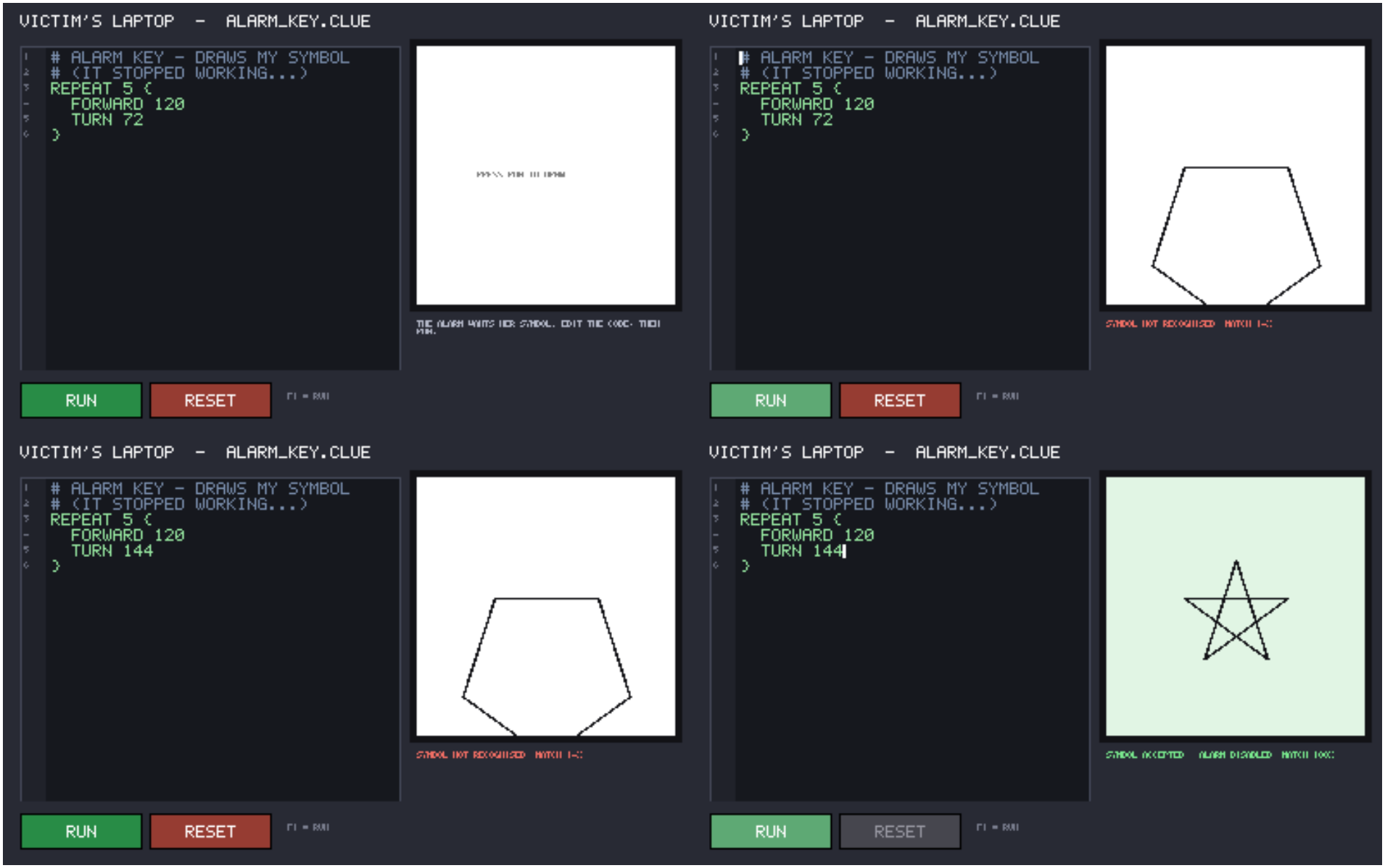
Sudheysi



Both drawings are rasterised with thick lines and compared with Intersection over Union. The starter code scores 14%, the correct star 100%, and the same star drawn by different code 93% (passes at 85%).

6. The victim's laptop: in-game code editor

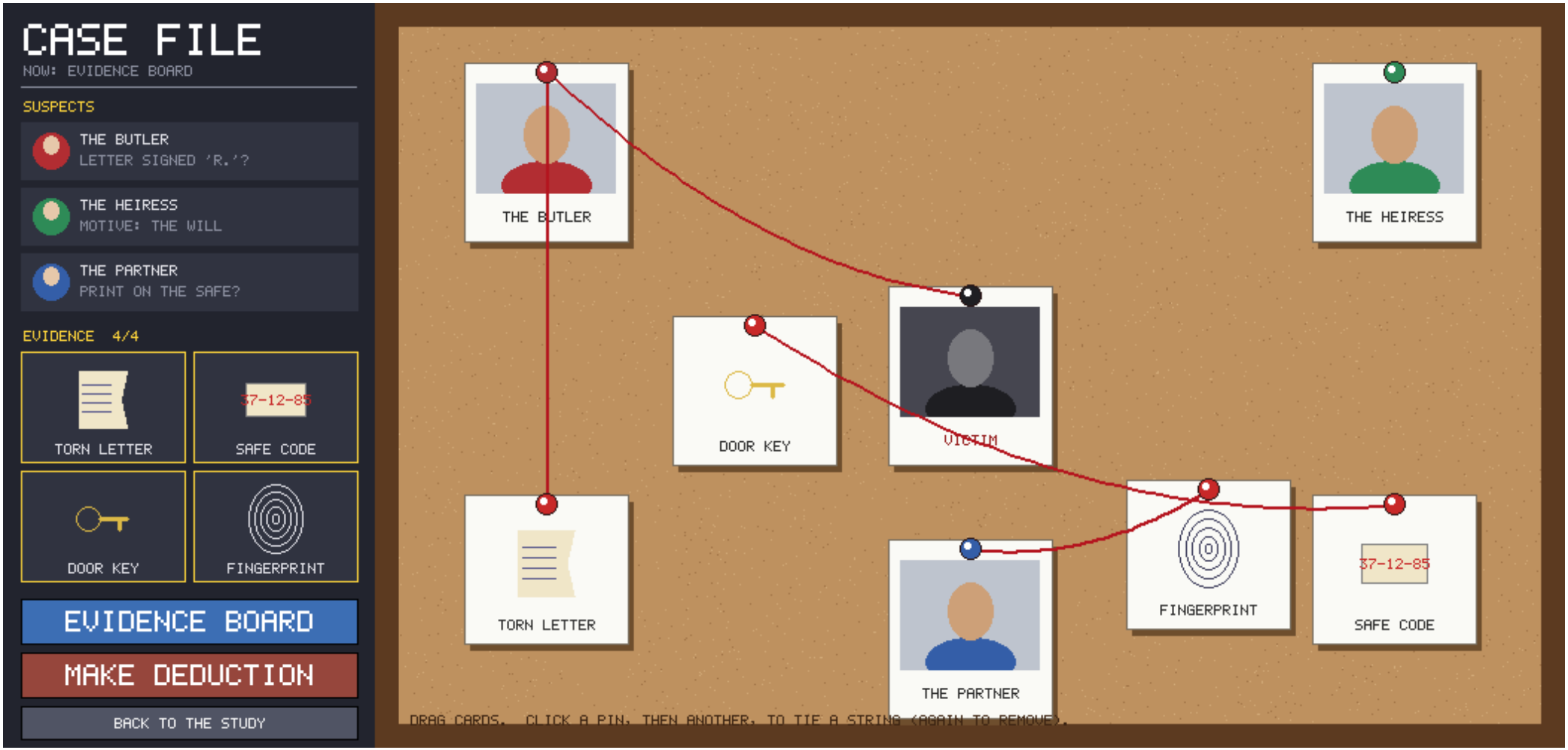
Latiffa



Monospace grid: row/column <-> pixel arithmetic places the cursor and highlights error lines. RUN runs our interpreter, draws the result in the preview monitor and checks it; the fixed program disables the safe's alarm.

7. Live Case File and interactive evidence board

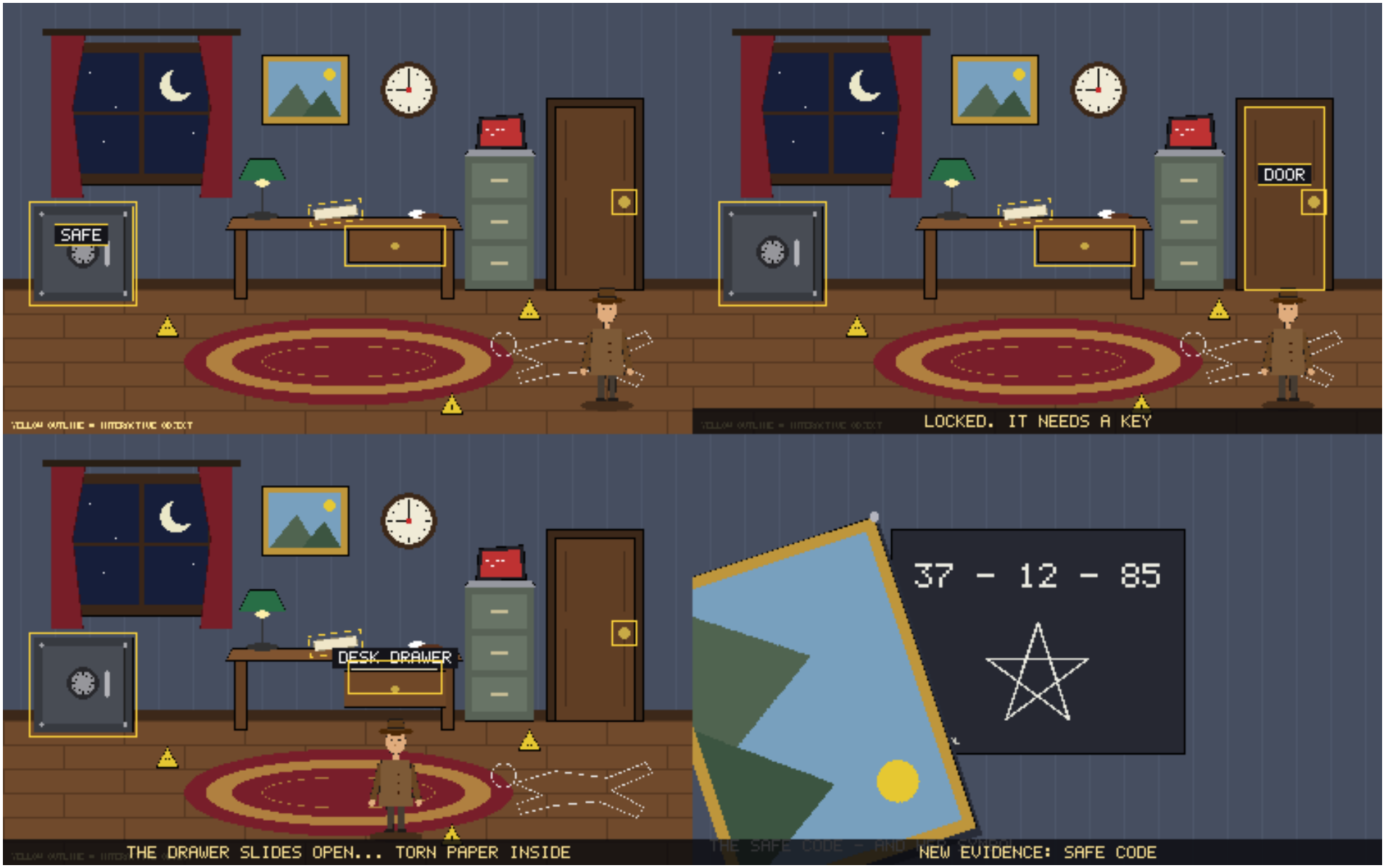
Latiffa



Evidence slots fill from the game state; board cards are dragged (grab offset + clamping) and pins are tied with sagging quadratic Bezier strings stored in the game state.

8. Playable crime scene

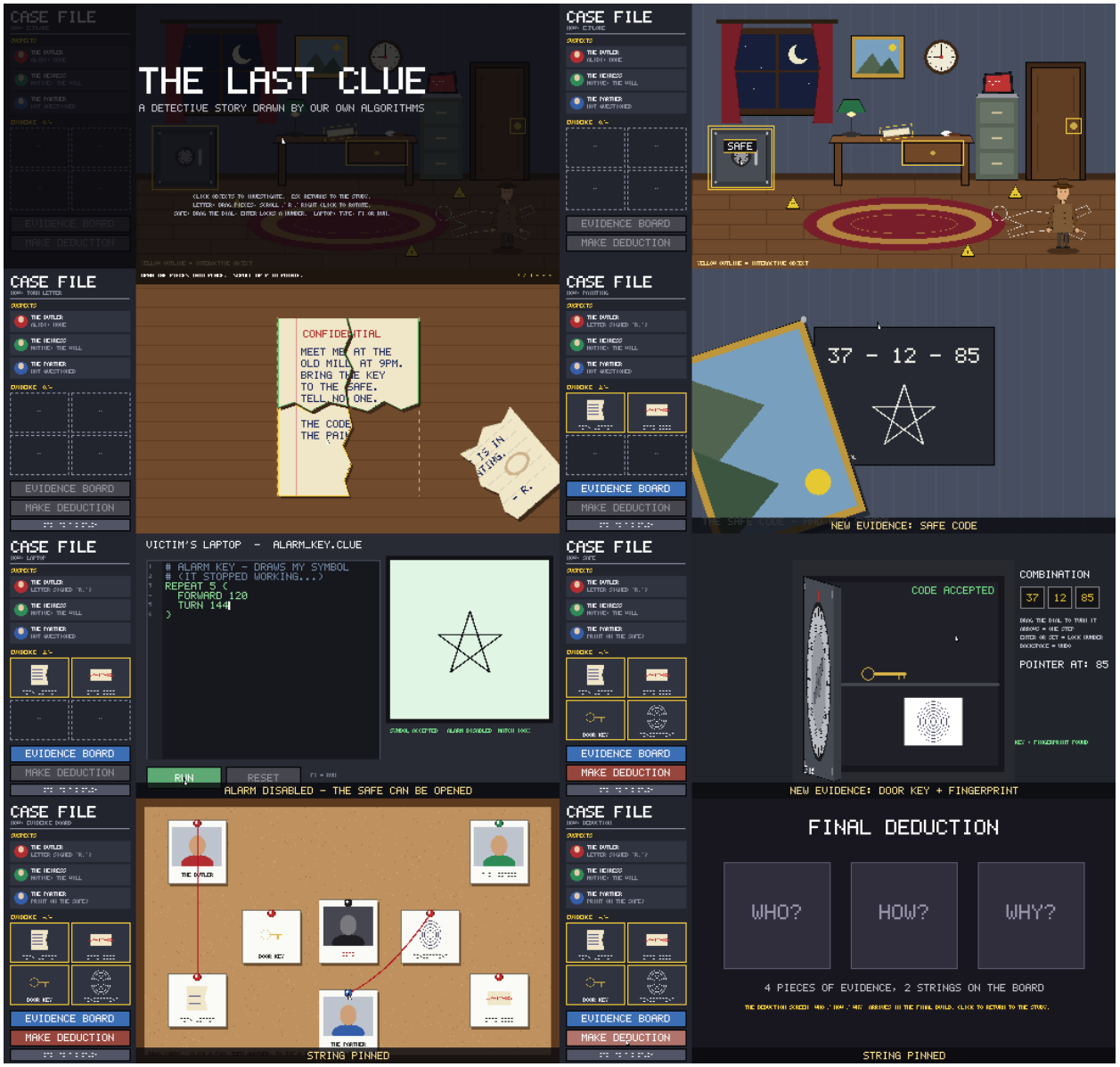
Alvin



Hotspot polygons with hover tags; the detective walks to the clicked object (translation over time + walk-cycle limbs) and its action runs; the painting close-up swings about its nail (rotation about a pivot, inverse mapping) to reveal the code and the symbol.

9. The whole prototype loop, played by a script

Caleb (integration)



Title -> drawer -> torn letter -> painting -> laptop code puzzle -> safe -> evidence board -> deduction, all through the same Game class the OpenGL window uses. Ends with 4/4 evidence, every puzzle solved and 2 strings on the board.